PROGRAMMING 2B (PROG6212)

Part 1

OTLOTLENG MABILETSA ST10478180

Contents

Section A — ERD 2

Section B — API Endpoint Plan 5

Section C — SQL Database Script 8

Section D — GitHub, CI/CD and Submission Evidence 17

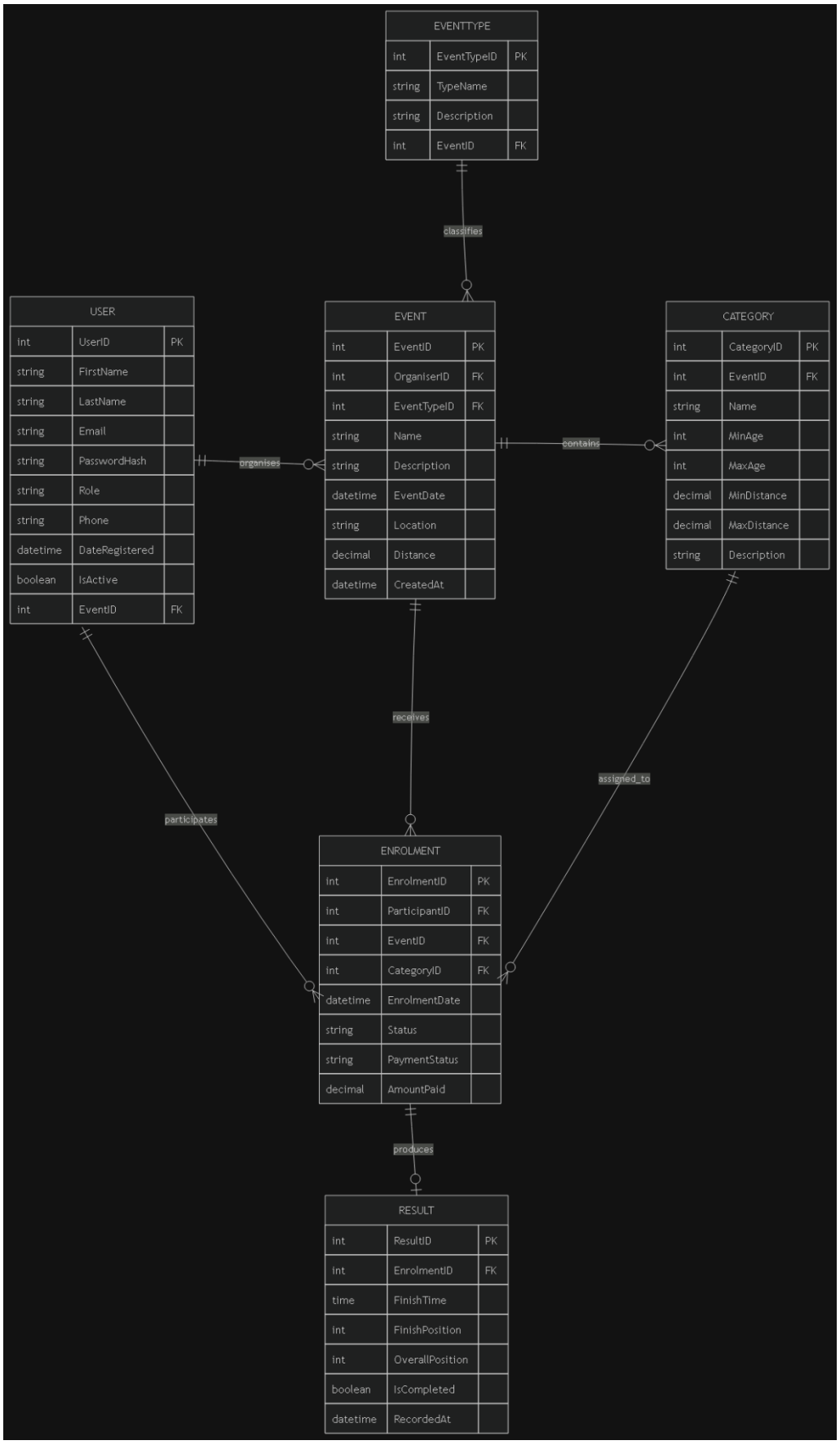
. GitHub Commit Evidence 19

References 20

Section A — ERD

Entity	Description
Users	Stores each user account and links the user to a role such as Organiser or Participant.
Roles	Lookup table for the two system roles: Organiser and Participant.
Events	Stores event information including name, description, date, location, distance and event type.
Categories	Stores age-based or distance-based categories used for event participation.
EventCategories	Junction table connecting Events and Categories in a many-to-many relationship.
Enrolments	Records Participants entering Events and selecting a Category.
Results	Stores a Participant result after an event, including finish time and finishing position.

ERD Screenshot



Relationship and Cardinality

Relationship	Cardinality	Explanation
Users → Roles	Many-to-One	Each user has one role; one role can belong to many users.
Roles → Users	One-to-Many	A role can be assigned to many user accounts.
Users → Enrolments	One-to-Many	A Participant can have many enrolments; each enrolment belongs to one participant.
Users → Events	One-to-Many	An Organiser can create many events; each event belongs to one Organiser.
Events → Enrolments	One-to-Many	One event can have many enrolments; each enrolment belongs to one event.
Events → EventCategories	One-to-Many	One event can be linked to many categories through EventCategories.
Categories → EventCategories	One-to-Many	One category can be linked to many events through EventCategories.
Enrolments → Results	One-to-One	Each enrolment can have zero or one result; each result belongs to one enrolment.
Enrolments → Categories	Many-to-One	Each enrolment selects one category; a category may be used by many enrolments.

. Primary and Foreign Keys

Entity	Primary Key	Foreign Key(s)
Users	UserId	RoleId → Roles.RoleId
Roles	RoleId	—
Events	EventId	OrganiserId → Users.UserId
Categories	CategoryId	—
EventCategories	EventCategoryId	EventId → Events.EventId; CategoryId → Categories.CategoryId
Enrolments	EnrolmentId	ParticipantId → Users.UserId; EventId → Events.EventId; CategoryId → Categories.CategoryId
Results	ResultId	EnrolmentId → Enrolments.EnrolmentId

Section B — API Endpoint Plan

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Register a new user account with role selection (Organiser or Participant).	None (Public)	{ email, password, firstName, lastName, role }	201 Created — userId, email, role; 400 Bad Request — validation error; 409 Conflict — email exists
POST	/api/auth/login	Authenticate a user and establish a session.	None (Public)	{ email, password }	200 OK — userId, email, role, firstName, lastName; 401 Unauthorized — invalid credentials
GET	/api/users/profile	Get the currently authenticated user profile.	Any (Logged in)	None	200 OK — user profile; 401 Unauthorized — not authenticated
PUT	/api/users/profile	Update the authenticated user profile.	Any (Logged in)	{ firstName, lastName, email }	200 OK — updated profile; 400 Bad Request — validation error; 401 Unauthorized
POST	/api/users/profile/picture	Upload a profile picture for the authenticated user.	Any (Logged in)	Multipart form data: { profilePicture }	200 OK — profilePictureUrl; 400 Bad Request — invalid file; 401 Unauthorized
GET	/api/events	Get all events with optional filters.	Any (Logged in)	None; query params: type, dateFrom, dateTo	200 OK — array of events; 401 Unauthorized
GET	/api/events/{id}	Get a specific event by ID including categories.	Any (Logged in)	None	200 OK — event object; 404 Not Found; 401 Unauthorized
POST	/api/events	Create a new event.	Organiser	{ name, description, eventDate, location, distance, eventType, bannerImage? }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden
PUT	/api/events/{id}	Update an existing event.	Organiser	{ name, description, eventDate, location, distance, eventType, bannerImage? }	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden — not owner; 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/events/{id}	Delete an event.	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden — not owner; 404 Not Found
POST	/api/events/{id}/banner	Upload an event banner image.	Organiser	Multipart form data: { bannerImage }	200 OK — bannerImageUrl; 400 Bad Request; 401 Unauthorized; 403 Forbidden
GET	/api/categories	Get all categories.	Any (Logged in)	None	200 OK — array of categories; 401 Unauthorized
GET	/api/categories/{id}	Get a specific category by ID.	Any (Logged in)	None	200 OK — category; 404 Not Found; 401 Unauthorized
POST	/api/categories	Create a new category.	Organiser	{ name, description, categoryType }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden
PUT	/api/categories/{id}	Update an existing category.	Organiser	{ name, description, categoryType }	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
DELETE	/api/categories/{id}	Delete a category.	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found
POST	/api/events/{eventId}/categories/{categoryId}	Add a category to an event.	Organiser	None	201 Created; 401 Unauthorized; 403 Forbidden — not owner; 404 Not Found
DELETE	/api/events/{eventId}/categories/{categoryId}	Remove a category from an event.	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden — not owner; 404 Not Found
GET	/api/enrolments	Get the authenticated participant enrolments or organiser event enrolments.	Any (Logged in)	None	200 OK — array of enrolments; 401 Unauthorized
GET	/api/enrolments/{id}	Get a specific enrolment.	Any (Logged in)	None	200 OK; 404 Not Found; 401 Unauthorized; 403 Forbidden
POST	/api/enrolments	Enrol a Participant in an event with a selected category.	Participant	{ eventId, categoryId }	201 Created; 400 Bad Request — validation/already enrolled; 401 Unauthorized; 403 Forbidden; 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
PUT	/api/enrolments/{id}/status	Update an enrolment status for an Organiser.	Organiser	{ status }	200 OK; 400 Bad Request — invalid status; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/events/{eventId}/enrolments	Get all enrolments for a specific event.	Organiser	None	200 OK — enrolments with participant details; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/results/my	Get all results for the authenticated Participant.	Participant	None	200 OK — result array with event details; 401 Unauthorized; 403 Forbidden
GET	/api/events/{eventId}/results	Get all results for a specific event.	Any (Logged in)	None	200 OK — result array; 401 Unauthorized; 404 Not Found
POST	/api/results	Capture a result for an enrolled Participant.	Organiser	{ enrolmentId, finishTime, finishingPosition }	201 Created; 400 Bad Request — validation/already exists; 401 Unauthorized; 403 Forbidden; 404 Not Found
PUT	/api/results/{id}	Update an existing result.	Organiser	{ finishTime, finishingPosition }	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
DELETE	/api/results/{id}	Delete a result.	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found

Section C — SQL Database Script

```
-- 1. CREATE DATABASE
CREATE DATABASE RaceDay;
GO
USE RaceDay;
GO

-- 2. CREATE TABLES
-- 2.1 Roles Table (Lookup)
CREATE TABLE Roles (
   RoleId INT IDENTITY(1,1) PRIMARY KEY,
   RoleName NVARCHAR(50) NOT NULL UNIQUE,
   Description NVARCHAR(200) NULL,
   CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.2 Users Table
CREATE TABLE Users (
   UserId INT IDENTITY(1,1) PRIMARY KEY,
   Email NVARCHAR(100) NOT NULL UNIQUE,
   PasswordHash NVARCHAR(255) NOT NULL,
   FirstName NVARCHAR(50) NOT NULL,
   LastName NVARCHAR(50) NOT NULL,
   RoleId INT NOT NULL,
   ProfilePictureUrl NVARCHAR(500) NULL,
   CreatedAt DATETIME DEFAULT GETDATE(),
   CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleId) REFERENCES Roles(RoleId),
   CONSTRAINT CHK_Users_Email CHECK (Email LIKE '%_@__%.__%')
);
GO

-- 2.3 Events Table
```

```
CREATE TABLE Events (
   EventId INT IDENTITY(1,1) PRIMARY KEY,
   OrganiserId INT NOT NULL,
   Name NVARCHAR(100) NOT NULL,
   Description NVARCHAR(500) NULL,
   EventDate DATETIME NOT NULL,
   Location NVARCHAR(200) NOT NULL,
   Distance DECIMAL(8,2) NOT NULL,
   EventType NVARCHAR(20) NOT NULL CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
   BannerImageUrl NVARCHAR(500) NULL,
   CreatedAt DATETIME DEFAULT GETDATE(),
   CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserId) REFERENCES Users(UserId)
);
GO

-- 2.4 Categories Table
CREATE TABLE Categories (
   CategoryId INT IDENTITY(1,1) PRIMARY KEY,
   Name NVARCHAR(50) NOT NULL,
   Description NVARCHAR(200) NULL,
   CategoryType NVARCHAR(20) NOT NULL CHECK (CategoryType IN ('Age', 'Distance')),
   CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.5 EventCategories Junction Table (Many-to-Many)
CREATE TABLE EventCategories (
   EventCategoryId INT IDENTITY(1,1) PRIMARY KEY,
   EventId INT NOT NULL,
   CategoryId INT NOT NULL,
   CONSTRAINT FK_EventCategories_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON
DELETE CASCADE,
   CONSTRAINT FK_EventCategories_Categories FOREIGN KEY (CategoryId) REFERENCES
Categories(CategoryId) ON DELETE CASCADE,
   CONSTRAINT UQ_EventCategories_EventCategory UNIQUE (EventId, CategoryId)
);
```

```
GO

-- 2.6 Enrolments Table
CREATE TABLE Enrolments (
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    EventId INT NOT NULL,
    CategoryId INT NOT NULL,
    EnrolmentDate DATETIME DEFAULT GETDATE(),
    Status NVARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled', 'Completed')),
    CONSTRAINT FK_Enrolments_Users FOREIGN KEY (ParticipantId) REFERENCES Users(UserId),
    CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON DELETE CASCADE,
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId),
    CONSTRAINT UQ_Enrolments_ParticipantEvent UNIQUE (ParticipantId, EventId)
);
GO

-- 2.7 Results Table
CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentId INT NOT NULL,
    FinishTime TIME NOT NULL,
    FinishingPosition INT NOT NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId) REFERENCES Enrolments(EnrolmentId) ON DELETE CASCADE,
    CONSTRAINT UQ_Results_Enrolment UNIQUE (EnrolmentId)
);
GO

-- 3. CREATE INDEXES FOR PERFORMANCE
CREATE INDEX IX_Users_RoleId ON Users(RoleId);
CREATE INDEX IX_Users_Email ON Users(Email);
CREATE INDEX IX_Events_OrganiserId ON Events(OrganiserId);
```

```
CREATE INDEX IX_Events_EventDate ON Events(EventDate);
CREATE INDEX IX_EventCategories_EventId ON EventCategories(EventId);
CREATE INDEX IX_EventCategories_CategoryId ON EventCategories(CategoryId);
CREATE INDEX IX_Enrolments_ParticipantId ON Enrolments(ParticipantId);
CREATE INDEX IX_Enrolments_EventId ON Enrolments(EventId);
CREATE INDEX IX_Enrolments_CategoryId ON Enrolments(CategoryId);
CREATE INDEX IX_Enrolments_Status ON Enrolments(Status);
CREATE INDEX IX_Results_EnrolmentId ON Results(EnrolmentId);
GO

-- 4. SEED DATA
-- 4.1 Seed Roles
INSERT INTO Roles (RoleName, Description) VALUES
('Organiser', 'Event organiser who can create and manage events, categories, and results'),
('Participant', 'Event participant who can browse events, enrol, and view results');
GO

-- 4.2 Seed Users
-- Placeholder password hashes are used for demonstration.
INSERT INTO Users (Email, PasswordHash, FirstName, LastName, RoleId, ProfilePictureUrl) VALUES
('thabo.mokoena@raceday.co.za', 'hashed_password_1', 'Thabo', 'Mokoena', 1, NULL),
('linda.zulu@raceday.co.za', 'hashed_password_2', 'Linda', 'Zulu', 1, NULL),
('sipho.ndlovu@gmail.com', 'hashed_password_3', 'Sipho', 'Ndlovu', 2, NULL),
('zanele.dlamini@gmail.com', 'hashed_password_4', 'Zanele', 'Dlamini', 2, NULL);
GO

-- 4.3 Seed Categories
INSERT INTO Categories (Name, Description, CategoryType) VALUES
('Under 20', 'Participants aged under 20 years', 'Age'),
('Senior (20-39)', 'Participants aged 20 to 39 years', 'Age'),
('Master (40+)', 'Participants aged 40 years and over', 'Age'),
('10km', '10 kilometre distance category', 'Distance'),
('21km', '21.1 kilometre half-marathon category', 'Distance'),
('42km', '42.2 kilometre marathon category', 'Distance');
GO
```

```
-- 4.4 Seed Events
INSERT INTO Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType, BannerImageUrl) VALUES
(1, 'Soweto Marathon', 'Annual Soweto Marathon through the streets of Soweto', '2026-09-15 06:00:00', 'Soweto, Johannesburg', 42.20, 'Run', NULL),
(1, 'Cape Town Cycle Tour', 'World-famous Cape Town Cycle Tour', '2026-10-20 07:00:00', 'Cape Town', 109.00, 'Cycle', NULL),
(2, 'Two Oceans Marathon', 'Ultra-marathon with stunning coastal views', '2026-11-05 05:30:00', 'Cape Town', 56.00, 'Run', NULL);
GO

-- 4.5 Link Events to Categories
INSERT INTO EventCategories (EventId, CategoryId) VALUES
(1, 1), (1, 2), (1, 3), (1, 6),
(2, 1), (2, 2), (2, 3),
(3, 1), (3, 2), (3, 3), (3, 5);
GO

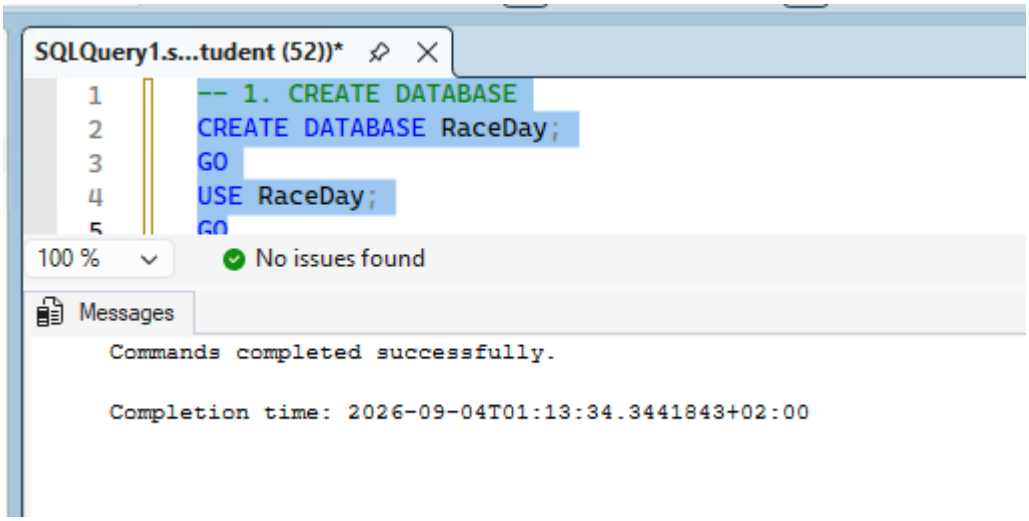
-- 4.6 Seed Enrolments
INSERT INTO Enrolments (ParticipantId, EventId, CategoryId, Status) VALUES
(3, 1, 2, 'Confirmed'),
(3, 2, 1, 'Pending'),
(4, 1, 3, 'Confirmed'),
(4, 3, 2, 'Pending');
GO

-- 4.7 Seed Results
INSERT INTO Results (EnrolmentId, FinishTime, FinishingPosition) VALUES
(1, '03:45:12', 47),
(3, '04:12:08', 89);
GO

-- 5. VERIFICATION QUERIES
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE';
GO
SELECT * FROM Roles;
SELECT * FROM Users;
```

```
SELECT * FROM Categories;
SELECT * FROM Events;
SELECT * FROM EventCategories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
GO
```

Create database



Create Tables

SQLQuery1.s...tudent (52))*

```sql
-- 2. CREATE TABLES
-- 2.1 Roles Table (Lookup)
CREATE TABLE Roles (
    RoleId INT IDENTITY(1,1) PRIMARY KEY,
    RoleName NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(200) NULL,
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.2 Users Table
CREATE TABLE Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    RoleId INT NOT NULL,
    ProfilePictureUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleId) REFERENCES Roles(RoleId),
    CONSTRAINT CHK_Users_Email CHECK (Email LIKE '%_@__%.__%')
);
GO

-- 2.3 Events Table
CREATE TABLE Events (
    EventId INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserId INT NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NULL,
    EventDate DATETIME NOT NULL,
    Location NVARCHAR(200) NOT NULL,
    Distance DECIMAL(8,2) NOT NULL,
    EventType NVARCHAR(20) NOT NULL CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
    BannerImageUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserId) REFERENCES Users(UserId)
);
GO
```

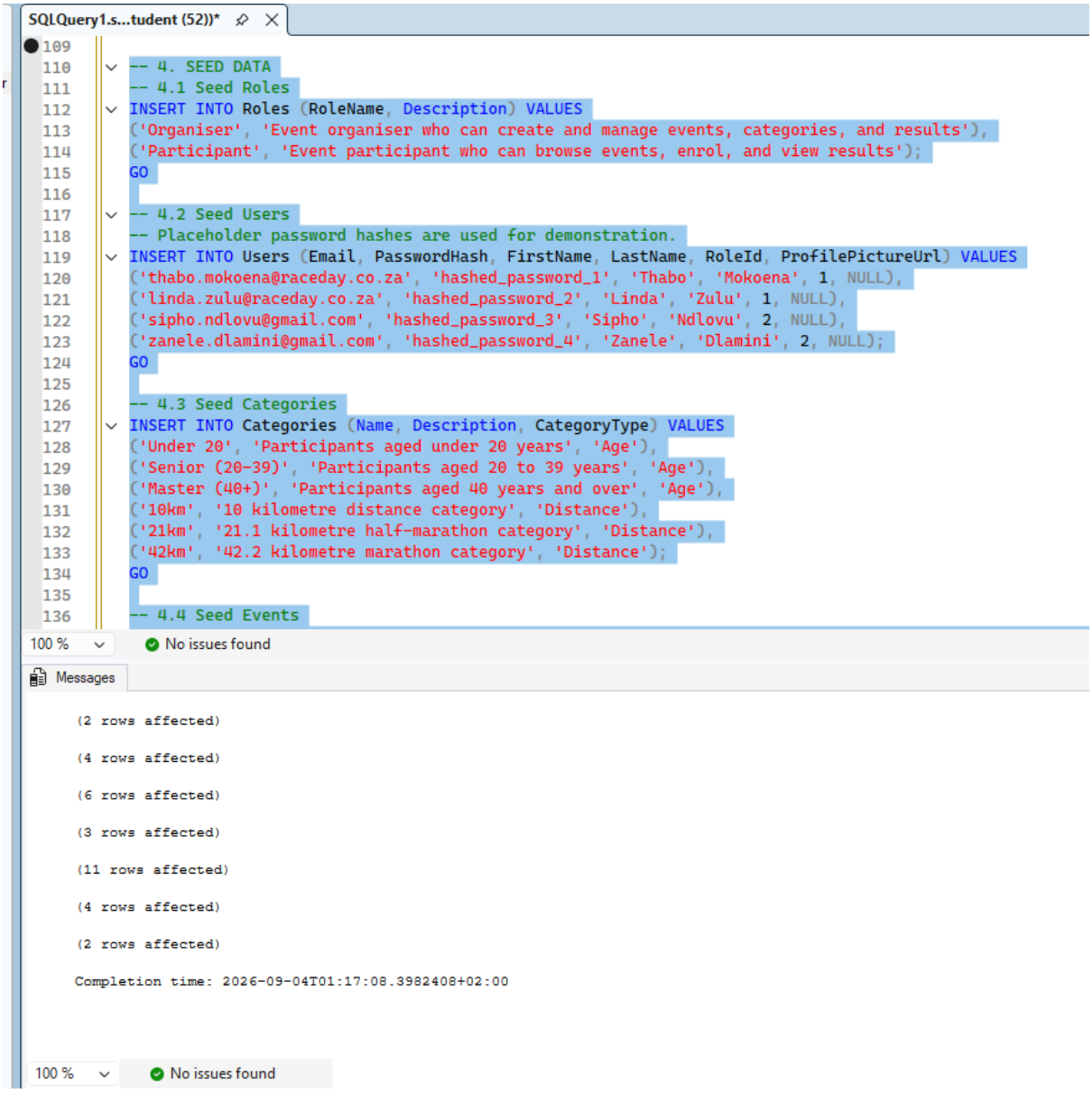
100 % No issues found

Messages

```
Commands completed successfully.

Completion time: 2026-09-04T01:15:24.7228111+02:00
```

Insert Data



```
SQLQuery1.s...tudent (52))*

109
110  -- 4. SEED DATA
111  -- 4.1 Seed Roles
112  INSERT INTO Roles (RoleName, Description) VALUES
113  ('Organiser', 'Event organiser who can create and manage events, categories, and results'),
114  ('Participant', 'Event participant who can browse events, enrol, and view results');
115  GO
116
117  -- 4.2 Seed Users
118  -- Placeholder password hashes are used for demonstration.
119  INSERT INTO Users (Email, PasswordHash, FirstName, LastName, RoleId, ProfilePictureUrl) VALUES
120  ('thabo.mokoena@raceday.co.za', 'hashed_password_1', 'Thabo', 'Mokoena', 1, NULL),
121  ('linda.zulu@raceday.co.za', 'hashed_password_2', 'Linda', 'Zulu', 1, NULL),
122  ('sipho.ndlovu@gmail.com', 'hashed_password_3', 'Sipho', 'Ndlovu', 2, NULL),
123  ('zanele.dlamini@gmail.com', 'hashed_password_4', 'Zanele', 'Dlamini', 2, NULL);
124  GO
125
126  -- 4.3 Seed Categories
127  INSERT INTO Categories (Name, Description, CategoryType) VALUES
128  ('Under 20', 'Participants aged under 20 years', 'Age'),
129  ('Senior (20-39)', 'Participants aged 20 to 39 years', 'Age'),
130  ('Master (40+)', 'Participants aged 40 years and over', 'Age'),
131  ('10km', '10 kilometre distance category', 'Distance'),
132  ('21km', '21.1 kilometre half-marathon category', 'Distance'),
133  ('42km', '42.2 kilometre marathon category', 'Distance');
134  GO
135
136  -- 4.4 Seed Events

100 %   No issues found

Messages

(2 rows affected)

(4 rows affected)

(6 rows affected)

(3 rows affected)

(11 rows affected)

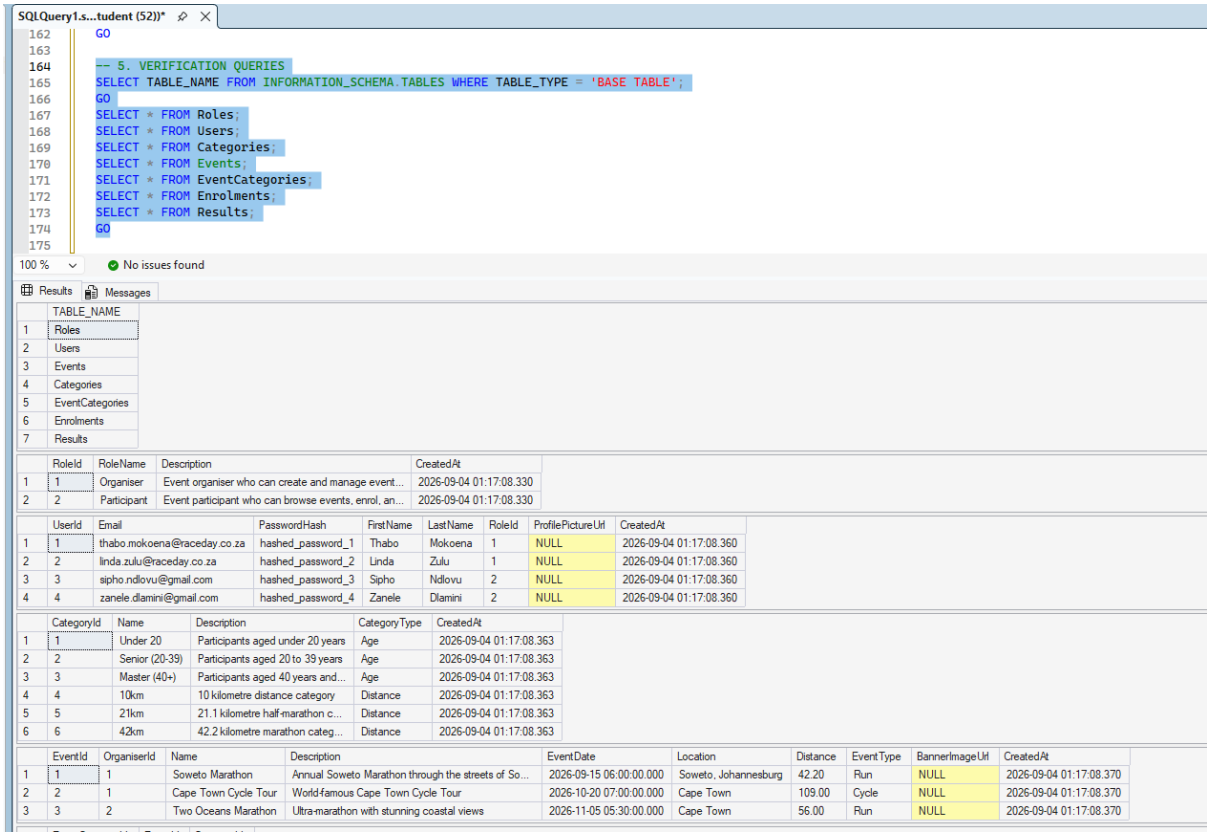
(4 rows affected)

(2 rows affected)

Completion time: 2026-09-04T01:17:08.3982408+02:00

100 %   No issues found
```

Verification Queries



SQLQuery1.s...tudent (52))*

```
GO

-- 5. VERIFICATION QUERIES
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE';
GO
SELECT * FROM Roles;
SELECT * FROM Users;
SELECT * FROM Categories;
SELECT * FROM Events;
SELECT * FROM EventCategories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
GO
```

100 % — No issues found

Results | Messages

	TABLE_NAME
1	Roles
2	Users
3	Events
4	Categories
5	EventCategories
6	Enrolments
7	Results

	RoleId	RoleName	Description	CreatedAt
1	1	Organiser	Event organiser who can create and manage event...	2026-09-04 01:17:08.330
2	2	Participant	Event participant who can browse events, enrol, an...	2026-09-04 01:17:08.330

	UserId	Email	PasswordHash	FirstName	LastName	RoleId	ProfilePictureUrl	CreatedAt
1	1	thabo.mokoena@raceday.co.za	hashed_password_1	Thabo	Mokoena	1	NULL	2026-09-04 01:17:08.360
2	2	linda.zulu@raceday.co.za	hashed_password_2	Linda	Zulu	1	NULL	2026-09-04 01:17:08.360
3	3	sipho.ndlovu@gmail.com	hashed_password_3	Sipho	Ndlovu	2	NULL	2026-09-04 01:17:08.360
4	4	zanele.dlamini@gmail.com	hashed_password_4	Zanele	Dlamini	2	NULL	2026-09-04 01:17:08.360

	CategoryId	Name	Description	CategoryType	CreatedAt
1	1	Under 20	Participants aged under 20 years	Age	2026-09-04 01:17:08.363
2	2	Senior (20-39)	Participants aged 20 to 39 years	Age	2026-09-04 01:17:08.363
3	3	Master (40+)	Participants aged 40 years and...	Age	2026-09-04 01:17:08.363
4	4	10km	10 kilometre distance category	Distance	2026-09-04 01:17:08.363
5	5	21km	21.1 kilometre half-marathon c...	Distance	2026-09-04 01:17:08.363
6	6	42km	42.2 kilometre marathon categ...	Distance	2026-09-04 01:17:08.363

	EventId	OrganiserId	Name	Description	EventDate	Location	Distance	EventType	BannerImageUrl	CreatedAt
1	1	1	Soweto Marathon	Annual Soweto Marathon through the streets of So...	2026-09-15 06:00:00.000	Soweto, Johannesburg	42.20	Run	NULL	2026-09-04 01:17:08.370
2	2	1	Cape Town Cycle Tour	World-famous Cape Town Cycle Tour	2026-10-20 07:00:00.000	Cape Town	109.00	Cycle	NULL	2026-09-04 01:17:08.370
3	3	2	Two Oceans Marathon	Ultra-marathon with stunning coastal views	2026-11-05 05:30:00.000	Cape Town	56.00	Run	NULL	2026-09-04 01:17:08.370

Section D — GitHub, CI/CD and Submission Evidence

The workflow should be stored in .github/workflows

```
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  validate-docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Check docs folder exists
        run: |
          if [ ! -d "docs" ]; then
            echo "Error: docs folder not found"
            exit 1
          fi

      - name: Check ERD file exists
        run: |
          if [ ! -f "docs/ERD.png" ] && [ ! -f "docs/ERD.pdf" ]; then
            echo "Error: ERD file not found"
            exit 1
          fi

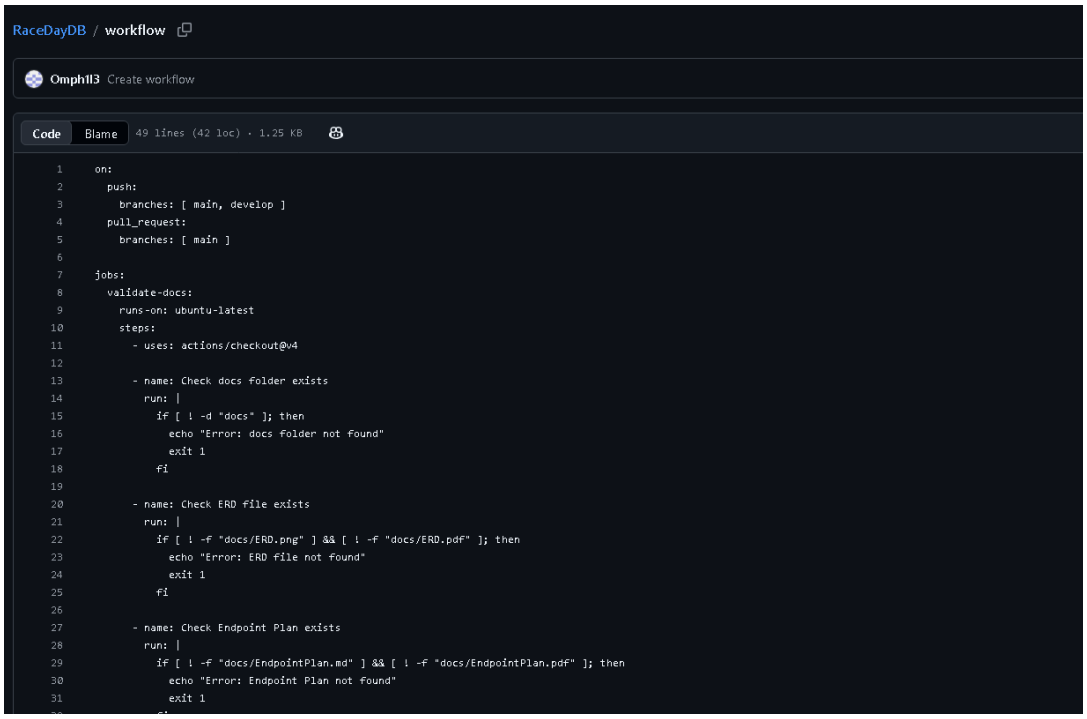
      - name: Check Endpoint Plan exists
        run: |
          if [ ! -f "docs/EndpointPlan.md" ] && [ ! -f "docs/EndpointPlan.pdf" ]; then
            echo "Error: Endpoint Plan not found"
```

```
      exit 1
    fi

- name: Check SQL Script exists
  run: |
    if [ ! -f "docs/RaceDayDB.sql" ]; then
      echo "Error: SQL script not found"
      exit 1
    fi

- name: Validate SQL script contains CREATE statements
  run: |
    if ! grep -q "CREATE TABLE" docs/RaceDayDB.sql; then
      echo "Error: SQL script missing CREATE TABLE statements"
      exit 1
    fi

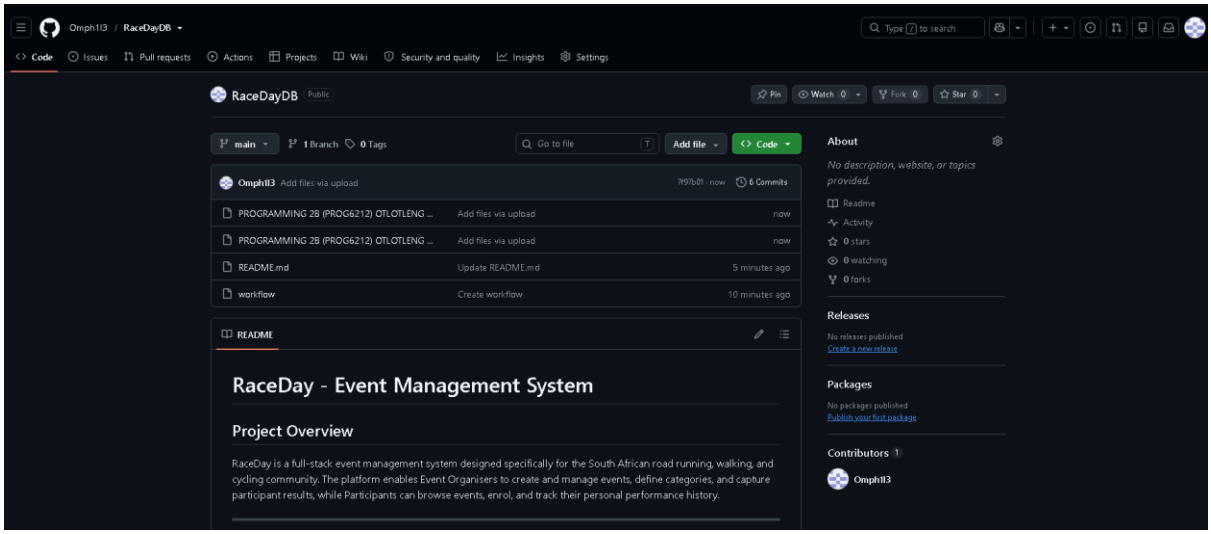
- name: Success
  run: echo "All Part 1 validation checks passed!"
```



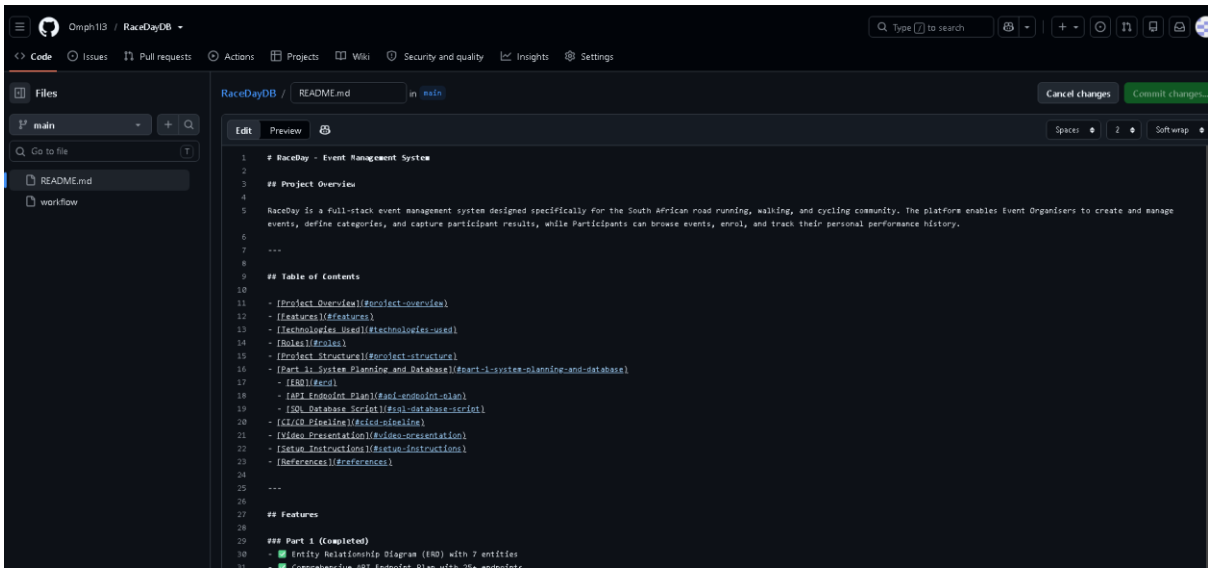
Screenshots

. GitHub Commit Evidence

https://github.com/Omph1l3/RaceDayDB



Readme



Screenshot

Video
link

References

3Wschool (2025). W3Schools.Com. [online] W3schools.com. Available at: https://www.w3schools.com/sql/ [Accessed 28 Aug. 2026].

Bootstrap (2023). Get started with Bootstrap. [online] getbootstrap.com. Available at: https://getbootstrap.com/docs/5.3/getting-started/introduction/ [Accessed 28 Aug. 2026].

chcomley (2026a). Azure Boards documentation. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/devops/boards/ [Accessed 28 Aug. 2026].

chcomley (2026b). Azure DevOps documentation. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/devops/ [Accessed 28 Aug. 2026].

dimitri-furman (2024). Monitoring and performance tuning — Azure SQL Database & Azure SQL Managed Instance. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/azure-sql/database/monitor-tune-overview [Accessed 28 Aug. 2026].

IBM (2024). Think topics | IBM. [online] IBM.com. Available at: https://www.ibm.com/think/topics/agile [Accessed 28 Aug. 2026].

pauljewellmsft (2023). Get started with Azure Blob Storage and .NET — Azure Storage. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-dotnet-get-started [Accessed 3 Sept. 2026].

scrumguides (2025). Home | Scrum Guides. [online] scrumguides.org. Available at: https://scrumguides.org/ [Accessed 28 Aug. 2026].